Projet Python 1SIO janvier 2025

Analyse de logs

Table des matières

1 - Organisation du projet....2
2 - Cahier des charges....2
2.1 - Contexte....2
2.2 - Structure des tables de la base de données logs_web....2
2.3 - Bonnes pratiques en matière de sécurité....2
2.4 - Exemple indicatif de processus de traitement....3
2.5 - Documents mis à disposition....3
2.6 - Environnement de travail....4
3 - Étape 1 : Python - Génération de pseudonymes à partir d'un fichier....4
4 - Étape 2 : Serveur Mariadb - Mise en place de la base de données en SQL....6
5 - Étape 3 : Recherche - Analyse des fichiers textes de login....7
6 - Étape 4 : SQL - Écriture d'une requête SQL pour valider un jeu d’essai....8
7 - Étape 5 : Python - Analyse des URL les plus consultées par utilisateur....8
8 - Étape 6 : Python - Extraction automatisée des logs vers un fichier CSV....8
9 - Étape 7 : Python - Génération de scripts SQL pour l'insertion de données....9
10 - Étape 8 – SQL - Exécution et test des scripts SQL d'insertion....9
11 - Étape 9 - Rédaction d’un mode opératoire....10
12 - Évolution 1 – Python - Génération automatique de scripts SQL....10
13 - Évolution 2 – Python - Analyse graphique des données des logs....11
Documentation à produire :....11
14 - Évolution 3 – SQL - Optimisation SQL pour des rapports détaillés....12
15 - Évolution 4 – Python / CSV - Analyse consolidée des logs....12
16 - Évolution 5 – Python - Archivage automatisé et sécurisé des journaux....13
17 - Évolution 6 – Python - Développement d'une interface graphique....14
18 - ANNEXE 1 - Structure des tables de la base de données logs_web....15

1 - Organisation du projet

Jour	Matin		Après-midi	Activité
Lun 06/01/25	10h - 12h00	atelier en anglais/CEJM	13h30 - 17h	Projet python
Mar 07/01/25	9h-12h30	Projet python	13h30 - 17h	Projet python
Mer 08/01/25	9h - 12h30	Projet python	13h30 - 17h	Projet python
Jeu 09/01/25	9h - 12h30	Auto-évaluation des projets	13h30 - 17h	blog + tableau des compétences
Ven 10/01/25	9h – 12h30	Rencontre avec les anciens		

2 - Cahier des charges

2.1 - Contexte

La société STESIO utilise un proxy pour enregistrer les accès au web réalisés par ses salariés, conformément aux obligations légales. Ces fichiers journaux contiennent des informations sensibles (adresses IP, dates, heures, et URL consultées), et leur volume important nécessite un traitement structuré. Pour cela, une base de données nommée **logs_web** sera créée pour stocker et structurer ces informations.**Objectifs :**

1. **Structurer les données** issues des fichiers journaux dans une base de données relationnelle.
2. **Faciliter l'analyse des logs** pour produire des statistiques, telles que :
   - Les sites les plus visités.
   - Les utilisateurs consommant le plus de ressources.
3. **Répondre à des besoins spécifiques**, comme :
   - Identifier les utilisateurs ayant accédé à un site donné à une date et une heure précises.

2.2 - Structure des tables de la base de données logs_web

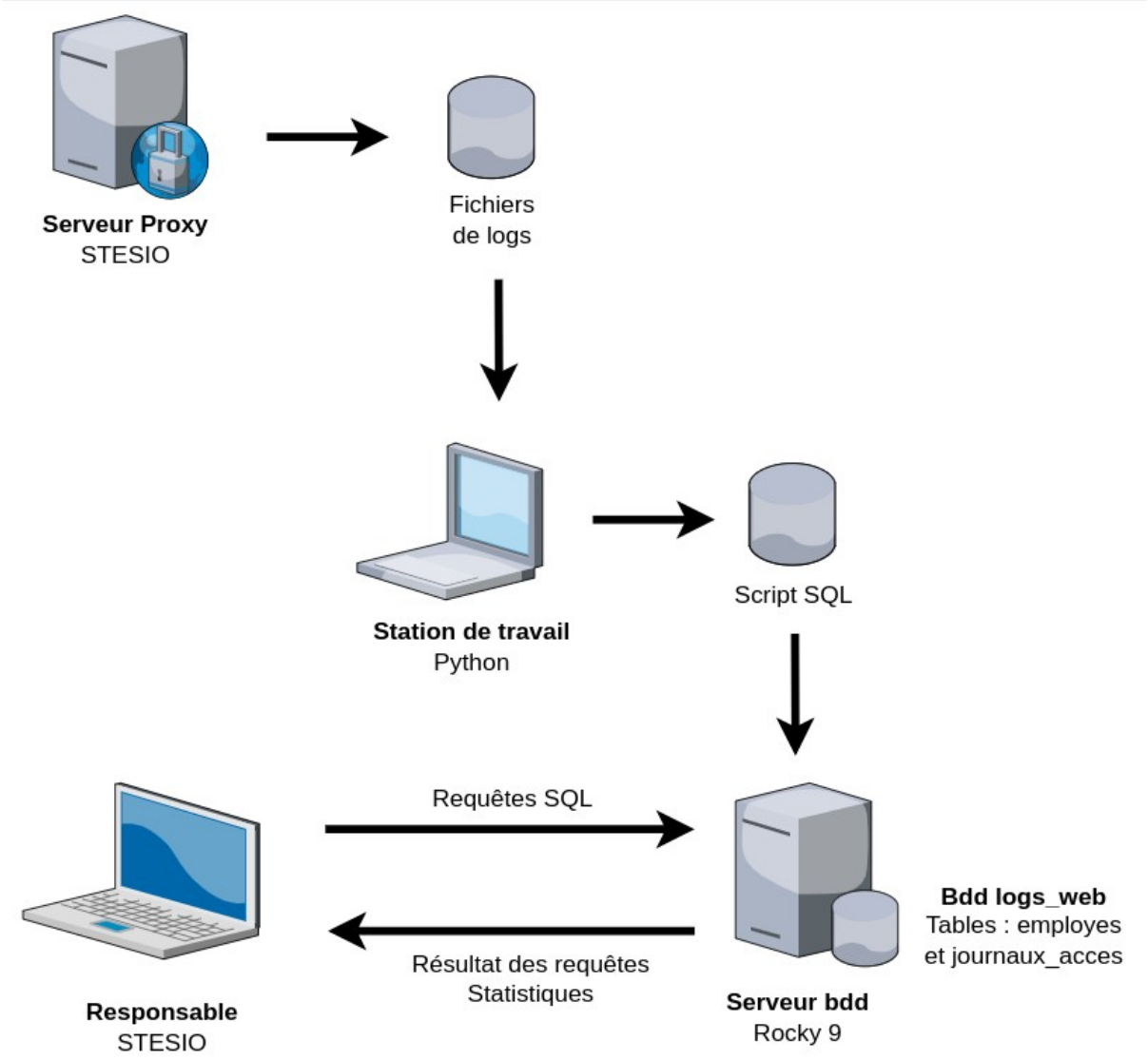
Voir ANNEXE 1

2.3 - Bonnes pratiques en matière de sécurité

Pour limiter les risques d'accès non autorisés, les bonnes pratiques suivantes sont mises en œuvre :

1. **Authentification et autorisation strictes :**
   - Création de deux utilisateurs distincts pour la gestion de la base de données :
     - .1 **Compte `utilog` :** Utilisateur avec des droits uniquement en lecture (`SELECT`) sur la base `logs_web`, permettant de consulter les données statistiques sans risque de modification accidentelle.
     - .2 **Compte `gestionlog` :** Utilisateur avec des droits complets (INSERT, UPDATE, DELETE) sur la base `logs_web`.
2. **Structure normalisée et relations :**
   - Les tables sont conçues avec des relations et contraintes claires pour garantir l'intégrité des données.

2.4 - Exemple indicatif de processus de traitement



2.5 - Documents mis à disposition

1. Le présent cahier des charges.
2. Jeux d'essai :
   - `log_proxy_2024-11-23.txt`
   - `log_proxy_2024-11-24.txt`
   - `log_proxy_2024-11-25.txt`
3. Documentation technique :
   - Gestion de fichiers et chaînes de caractères en Python.
4. Grille d'évaluation du projet (pour la recette finale).

2.6 - Environnement de travail

Vous pourrez développer et tester vos scripts Python dans l'environnement de votre choix, tel que Spyder, Visual Studio Code, ou NetBeans, directement sur votre station de travail.

Une machine virtuelle sous Rocky Linux 9 avec les services Apache, MariaDB, et phpMyAdmin sera mise à votre disposition. Cette VM (serveur web/mariadb) sera spécifiquement utilisée pour tester vos scripts SQL et vérifier leur bon fonctionnement.

3 - Étape 1 : Python - Génération de pseudonymes à partir d'un fichier

Dans cette étape, vous allez lire un fichier (fichier d'entrée) contenant des informations sur des personnalités célèbres, effectuer des transformations pour générer des pseudonymes au format **pnomagecode**, calculer leur âge en **2025**, et écrire les résultats dans un nouveau fichier (fichier de sortie). Vous structurerez votre programme en créant des **fonctions dédiées** pour chaque opération, et vous documenterez vos résultats. Tous les pseudonymes générés doivent être écrits en **minuscules**.

Compréhension globale du programme :

Il est essentiel de comprendre les principes suivants :

- **Lecture du fichier** : Le contenu du fichier d'entrée est chargé dans une structure de données (une **liste**) en mémoire.
- **Traitement des données** : Les transformations sont effectuées directement sur cette liste en mémoire.
- **Écriture du fichier de sortie** : Une fois toutes les transformations réalisées, l'ensemble des résultats est passé à la fonction d'écriture pour être enregistré dans le fichier de sortie.

Cette approche évite de réaliser des opérations répétées de lecture et d'écriture sur le disque, qui sont coûteuses en ressources. Le fichier est donc : **lu une seule fois**, **traité en mémoire**, **écrit une seule fois**.

Tâches à réaliser :

a - Création du fichier d'entrée

- Créez un fichier nommé **testEntree.txt** contenant les informations suivantes :

```
Emile;ZOLA;1840
Victor;HUGO;1802
George;SAND;1804
Jane;AUSTEN;1775
Mark;TWAIN;1835
```

b - Création d'un script python

- Rédigez un script Python nommé **pseudo.py** qui réalise les opérations suivantes :

1. **Lecture des données**
   - Lire le fichier **testEntree.txt** en mode lecture.
   - Implémentez une fonction spécifique qui ouvre le fichier et charge son contenu dans une liste où chaque ligne représente une personnalité.
2. **Traitement des données**
   - Pour chaque ligne du fichier d'entrée, effectuez les transformations suivantes :
     - **Extraction des données** : Séparez le prénom, le nom et l'année de naissance.
     - Implémentez des fonctions dédiées pour :
       1. **Calculer l'âge** : Soustrayez l'année de naissance de l'année actuelle (**2025**).
       2. **Générer un pseudonyme** au format suivant :
          - **Première lettre** du prénom, en minuscule.
          - **Trois premières lettres** du nom, en minuscule.
          - **Âge** calculé.
          - **Nombre aléatoire** compris entre **10** et **99**.

   Exemple de transformations :

   Ligne d'entrée : `George;SAND;1804`
   - Prénom = George → `g`
   - Nom = SAND → `san`
   - Âge = 2025 - 1804 = `221`
   - Nombre aléatoire = `35`
   - **Pseudonyme généré** : `gsan22135`
3. **Écriture des résultats**
   - Implémentez une fonction pour écrire les résultats dans un fichier nommé **testSortie.txt**.
   - Chaque pseudonyme doit être écrit sur une ligne séparée et en minuscules.

   Exemple de contenu attendu dans le fichier testSortie.txt après exécution :

```
ezol18542
vhug22387
gsan22135
...
```

Document à produire :

1. **Compte-rendu détaillé** :
   - Expliquez les transformations effectuées et ajoutez des captures d'écran illustrant chaque étape, avec des explications.
2. **Code source Python commenté** :
   - Fournissez le programme complet **pseudo.py** avec des commentaires détaillant les fonctions et les étapes clés.
3. **Contenus des fichiers** :
   - **testEntree.txt** : Affichez le contenu initial avec les années de naissance.
   - **testSortie.txt** : Affichez le contenu généré après l'exécution du programme.

4 - Étape 2 : Serveur Mariadb - Mise en place de la base de données en SQL

Dans cette étape, vous allez mettre en place une base de données MariaDB permettant à la société STESIO de gérer efficacement les données liées à l'utilisation de leur proxy. Cette base sera utilisée pour journaliser les accès au web réalisés par les salariés, répondre aux obligations légales et permettre l'analyse des données à des fins statistiques. Vous devrez créer les tables nécessaires, les remplir avec les données fournies, et vérifier la connectivité locale et distante du système.

- Lancez votre VM `votrelogin_SRVWEB_T_rocky9.5_mariadb_apache_phpmyadmin-01-01-2025_1x_PIS` (x valant A ou B)

  Chemin : DatacenterSIO > 1SIO > 1SIOA (ou B) > Projet_Inter_Sem_1SIOA (ou B)
- Renommez votre machine rocky-NNNNP.btssioXXX.lan (uniquement le nom de votre machine sous l'OS Rocky, **ne changez pas le nom de votre machine sous VMWare**).

  NNNN : 4 premières lettres de votre nom P : 1ère lettre de votre prénom

  XXX : valeur numérique donnée individuellement sur le plan d'adressage IP fourni sur Moodle
- Mettez votre VM dans le bon VLAN et configurez les paramètres IP suivant le plan d'adressage IP donné sur Moodle.
- Créez une base de données `logs_web` avec comme jeu de caractère `utf8` et interclassement `utf8_unicode_ci` à l'aide d'une requête SQL. Compte *root* mdp *joliverie*.
- Créez les utilisateurs MariaDB suivants à l'aide de requêtes SQL :
  - **utilog** : Compte avec un accès en lecture seule (**SELECT**) pour consulter les statistiques en toute sécurité sur la base de données `logs_web`.
  - **gestionlog** : Compte avec un accès complet (**INSERT, UPDATE, DELETE, SELECT**) pour l'administration des données sur la base de données `logs_web`.
- Créez les tables requises et insérez les données fournies dans le cahier des charges en utilisant des requêtes SQL.

- Testez en local et à distance que le compte **gestionlog** peut gérer la base de données. Pour le test à distance, procédez comme suit :
  - Modifiez le fichier *mariadb-server.cnf* du serveur mariadb pour remplacer *BIND-ADDRESS= 127.0.0.1* par *BIND-ADDRESS=@IP (adresse IP du serveur MariaDB* : vous paramétrez ainsi l'écoute de votre serveur sur l'interface réseau de votre machine).
  - Redémarrez le service `mariadbd.` Vérifier que ce service est en écoute sur l'adresse IP de votre serveur MariaDB et sur le port 3306.
  - Modifiez le fichier *phpMyAdmin.conf* du serveur mariadb pour ajouter sous *Require local* l'option suivante *: Require ip* ***@IP de votre machine distante***

    Redémarrez le service `httpd`.

    Ainsi l'accès à `phpMyAdmin` est autorisé en local mais aussi seulement, pour des questions de sécurité, depuis la machine distante du responsable.
- Fournissez un exemple de requête SQL pour insérer un enregistrement dans la table `journaux_acces`, en respectant le format des dates et en laissant l'identifiant (id_acces) s'auto-incrémenter.
- Testez cette requête pour vérifier son bon fonctionnement, puis supprimez l'enregistrement ajouté.

Documentation à produire :

1. **Compte-rendu des opérations** :
   - Résumez les étapes réalisées et les résultats obtenus, en incluant des **captures d'écran commentées**.
   - Détaillez :
     - Les ordres SQL pour créer la base de données, les utilisateurs avec leurs droits, et les deux tables, accompagnés d'explications.
     - Un exemple de requête SQL pour insérer un enregistrement dans la table **PROXY**.
2. **Schéma relationnel** :
   - Fournissez un schéma relationnel de la base de données généré avec le concepteur **phpMyAdmin**.
3. **Captures d'écran** :
   - Les bases de données accessibles par le compte **gestionlog**.
   - Les droits du compte **gestionlog** sur la base de données.
   - La connexion réussie au serveur MariaDB à distance avec le compte **gestionlog**.

5 - Étape 3 : Recherche - Analyse des fichiers textes de login

- Analyser les fichiers texte au format `log_proxy_yyyy-mm-dd.txt` pour identifier les informations disponibles sur chaque ligne et déterminer à quels champs de la base de données ces informations peuvent correspondre. Cette étape permettra d'assurer une correspondance précise entre les données extraites des logs et les colonnes de la base de données.

Documentation à produire :

- **Document d'analyse détaillée :**
  - Décrivez chaque élément présent dans les logs.
  - Expliquez à quels champs de la base de données ces éléments correspondent.
  - Incluez un tableau récapitulatif pour clarifier les correspondances.
  - Mentionnez les informations qui ne sont pas utilisées dans cette étape, avec une justification.

6 - Étape 4 : SQL - Écriture d'une requête SQL pour valider un jeu d'essai

- Rédigez une requête SQL permettant d'afficher, pour chaque utilisateur, l'URL la plus visitée accompagnée de son nombre de visites. Cette requête devra être rédigée mais ne sera pas testée dans cette étape.

Documentation à produire :

- Description de la requête : Présentation des étapes de conception et explication du fonctionnement de la requête SQL.
- Code SQL complet : Fournir la requête permettant d'atteindre l'objectif demandé.
- Exemple de résultat : Proposer un exemple illustrant les données retournées par la requête.

7 - Étape 5 : Python - Analyse des URL les plus consultées par utilisateur

- Réalisez un programme Python, nommé `url_par_utilisateur.py`, qui affiche les fichiers logs disponibles dans un dossier sous forme de menu.

L'utilisateur sélectionnera un fichier à analyser en saisissant le numéro correspondant. Le programme analysera ensuite le fichier sélectionné pour identifier, pour chaque utilisateur, l'URL la plus visitée et son nombre de visites. Structuré de manière modulaire, le programme doit utiliser des fonctions adaptées pour chaque étape du traitement. Le résultat final doit présenter, pour chaque utilisateur, l'URL la plus visitée avec son nombre de visites, être stocké dans une variable appropriée, et affiché à l'écran.

Documentation à produire :

- Compte-rendu : Résumé des étapes d'analyse et des résultats obtenus, avec captures d'écran annotées.
- Code source Python : Programme `url_par_utilisateur.py`, structuré et commenté.
- Résultats finaux : Liste des utilisateurs avec leur URL la plus visitée et le nombre de visites.

8 - Étape 6 : Python - Extraction automatisée des logs vers un fichier CSV

- Développez un programme Python nommé `logproxy.py` qui lit un fichier de log en fonction d'une date saisie par l'utilisateur au format `YYYY-MM-DD`. Ce programme doit extraire les informations nécessaires pour compléter les champs de la table `acces_logs` de la base de données. Les données extraites doivent être enregistrées dans un fichier CSV nommé `loginfo.csv`, structuré conformément aux normes des fichiers CSV. Le fichier doit inclure une première ligne d'en-tête au format suivant :

  `date;heure;adresse_ip_employe;url_consultee;methode_http;code_reponse.`

Les données doivent être correctement formatées et séparées par des points-virgules (`;`). L'utilisation d'une bibliothèque adaptée, telle que `csv`, est obligatoire pour garantir un formatage correct, et la gestion brute de fichiers TXT est interdite.

Documentation à produire :

- Compte-rendu : Description des opérations, résultats obtenus, et captures d'écran annotées.
- Code source Python : Code de `logproxy.py`, structuré et commenté.
- Fichier CSV : Exemple du fichier `loginfo.csv` généré, conforme au format spécifié.

9 - Étape 7 : Python - Génération de scripts SQL pour l'insertion de données

Phase 1 : Génération individuelle des scripts SQL

- Réalisez une première version du programme Python permettant de sélectionner un fichier log parmi ceux disponibles dans un dossier. Le programme doit afficher la liste des fichiers logs sous forme de menu interactif, où chaque fichier est associé à un numéro. L'utilisateur doit pouvoir choisir un fichier en saisissant le numéro correspondant.

  Le programme, nommé **`generate_sql_insert.py`**, doit ensuite analyser le contenu du fichier log sélectionné et générer un fichier SQL dont le nom est basé sur la date présente dans le nom du fichier log. Le fichier SQL doit être nommé au format **`insert_log_yyyy-mm-dd.sql`**, où `yyyy-mm-dd` correspond à la date du fichier log sélectionné (par exemple : pour le fichier log **`log_proxy_2025-01-03.txt`**, le fichier SQL généré sera nommé **`insert_log_2025-01-03.sql`**). Ce fichier SQL doit contenir les commandes nécessaires pour insérer les enregistrements du log dans la table **`acces_logs`**.

  Cette opération doit être répétée manuellement pour chaque fichier log : à chaque exécution du programme, l'utilisateur sélectionne un fichier log, et un nouveau fichier SQL est généré avec le nom correspondant à la date du fichier log.

  Veillez à structurer votre programme de manière claire en séparant la lecture du fichier log, l'analyse des données et la génération du fichier SQL dans des fonctions distinctes. Ajoutez des commentaires pour expliquer chaque étape de votre code.

Phase 2 : Génération automatique pour tous les fichiers logs (à faire dans l'Évolution 1)

Dans l'évolution 1, il vous sera demandé d'améliorer le programme pour automatiser l'analyse et la génération des fichiers SQL pour l'ensemble des fichiers logs présents dans le dossier. Lors de l'exécution, le programme devra parcourir tous les fichiers logs et générer un fichier SQL distinct pour chacun d'entre eux, sans intervention manuelle pour chaque fichier.

Documentation à produire :

- **Compte-rendu** : Description des opérations réalisées avec captures d'écran annotées (menu de sélection, analyse du fichier log, génération du fichier SQL).
- **Code source Python** : Code complet complet **generate_sql_insert.py**, structuré et commenté.
- **Exemple de fichier SQL** : Un exemple de fichier **insert_log_yyyy-mm-dd.sql** généré à partir d'un fichier log.

10 - Étape 8 – SQL - Exécution et test des scripts SQL d'insertion

- Exécutez les scripts SQL générés à l'étape précédente pour insérer les données dans la table **acces_logs**. Vérifiez ensuite que la table a bien été remplie en effectuant des contrôles appropriés. Enfin, testez la requête SQL élaborée à l'étape 4 pour valider son fonctionnement avec les données insérées.

Documents à produire :

- **Compte-rendu** : Description des étapes d'exécution, vérification de la table **acces_logs**, et résultats des tests de la requête SQL, avec captures d'écran annotées.
- **Exemple de script SQL** : Un script utilisé pour l'insertion.
- **Résultats de la requête** : Preuve de l'exécution correcte de la requête, via capture ou export.

11 - Étape 9 - Rédaction d'un mode opératoire

Vous allez faire un mode opératoire pour guider un utilisateur dans l'insertion des données des fichiers logs dans la base de données. Ce document expliquera comment utiliser le programme Python pour générer les scripts SQL, exécuter ces scripts pour insérer les données, et vérifier leur intégration avec des requêtes de test.

- Rédigez un mode opératoire à destination de l'administrateur système. Ce document devra inclure les points suivants :
  1. **Présentation du format des fichiers logs** :
     - Description de la structure des fichiers logs (champs, séparateurs, exemples de contenu).
  2. **Utilisation du programme Python** :
     - Étapes pour lancer le programme Python.
     - Explication du menu interactif pour sélectionner un fichier log.
     - Génération automatique des scripts SQL et leur emplacement de sauvegarde.
  3. **Exécution des scripts SQL** :
     - Instructions pour exécuter les scripts SQL générés.
     - Vérification que les données ont bien été insérées dans la table `acces_logs`.
  4. **Test des requêtes SQL** :
     - Étapes pour exécuter les requêtes de test et valider les données insérées.
     - Analyse des résultats obtenus pour confirmer la conformité des données.

Documentation à produire :

- **Mode opératoire** : Document détaillé et structuré expliquant le processus complet, depuis l'utilisation du programme Python jusqu'à la validation des données insérées.
- **Exemples** : Exemple de fichier log, script SQL généré, et résultat d'une requête de test pour valider les opérations.

12 - Évolution 1 – Python - Génération automatique de scripts SQL

- Réalisez un programme Python nommé **`generate_sql_batch.py`** qui lit automatiquement tous les fichiers logs présents dans un dossier donné et les traite en une seule exécution. Le programme doit parcourir l'ensemble des fichiers logs, lire leur contenu, analyser les données et générer un fichier SQL distinct pour chaque fichier log.

  Chaque fichier SQL doit porter un nom explicite au format **`insert_log_yyyy-mm-dd.sql`**, où **`yyyy-mm-dd`** correspond à la date présente dans le nom du fichier log (par exemple : **`log_proxy_2025-01-03.txt`** → **`insert_log_2025-01-03.sql`**). Le fichier SQL doit contenir les commandes nécessaires pour insérer les enregistrements du log dans la table **`acces_logs`**.

  Le programme doit être structuré en plusieurs fonctions distinctes pour garantir une meilleure lisibilité et modularité.

  Veillez à documenter votre code avec des commentaires expliquant les étapes clés du traitement pour rendre le programme clair et compréhensible.

Documentation à produire :

- **Compte-rendu** : Résumé des étapes de traitement et résultats obtenus, avec captures d'écran annotées.
- **Code source Python** : Script complet `generate_all_sql.py`, structuré et commenté.
- **Exemples de fichiers SQL** : Deux exemples de fichiers SQL générés.

13 - Évolution 2 – Python - Analyse graphique des données des logs

- Réalisez un programme Python, nommé `analyse_graphique.py`, qui génère des visualisations pour mieux comprendre les données issues des fichiers logs. Ce programme devra :
  1. Charger et analyser les données des fichiers logs ou de la base de données.
  2. Produire des graphiques pour :
     - Les URL les plus visitées avec leur nombre total de visites.
     - Les employés les plus actifs selon le nombre de visites.
     - La répartition des codes HTTP (200, 404, etc.).
     - L'activité par plage horaire (par heure ou par jour).
  3. Permettre l'enregistrement des graphiques sous forme d'images ou fichiers interactifs HTML.

Le programme doit être structuré de manière modulaire, en utilisant des fonctions dédiées pour chaque étape (chargement des données, traitement, génération des graphiques).

Aide pour les bibliothèques Python :

Pour réaliser ce programme, vous pouvez utiliser les bibliothèques suivantes :

- **Pandas** : pour charger, manipuler et analyser les données.
- **Matplotlib** : pour générer des graphiques simples et statiques.
- **Seaborn** : pour des graphiques esthétiques et adaptés aux données tabulaires.
- **Plotly** ou **Dash** : pour des visualisations interactives et dynamiques avec des filtres.

Documentation à produire :

- **Compte-rendu** : Description des graphiques produits et de leur utilité. Résultats obtenus, avec captures d'écran annotées des visualisations.
- **Code source Python** : Script complet `analyse_graphique.py`, structuré et commenté pour expliquer chaque étape.
- **Exemples de graphiques** : Un ou deux exemples illustrant les données visualisées, avec des explications des tendances ou anomalies détectées.

14 - Évolution 3 – SQL - Optimisation SQL pour des rapports détaillés

- Réalisez une série de requêtes SQL optimisées pour générer des rapports détaillés à partir de la table `acces_logs`. Ces rapports devront permettre de répondre aux questions suivantes :

  1. **Activité horaire et quotidienne :** Identifier les périodes de forte activité en regroupant les données par heure et par jour.
  2. **URLs problématiques :** Lister les URLs générant le plus d'erreurs HTTP (codes 404, 500, etc.), triées par ordre décroissant de fréquence.
  3. **Répartition des méthodes HTTP :** Calculer la proportion des différentes méthodes HTTP utilisées (GET, POST, etc.).
  4. **Top des employés les plus actifs par jour :** Identifier, pour chaque jour, les employés ayant effectué le plus grand nombre de requêtes.
  5. **Sites consultés par au moins 3 employés différents :** Lister les URLs ayant été visitées par au moins trois employés distincts.

  Les requêtes devront être écrites de manière efficace, en exploitant les fonctions SQL disponibles (ex. : `GROUP BY`, `COUNT`, `SUM`, `CASE`, `HAVING`, sous-requêtes, etc.). Une attention particulière devra être portée à la clarté et à l'optimisation des requêtes pour garantir leur performance.

Documentation à produire :

- **Compte-rendu** : Résumé des questions traitées par les requêtes. Description des résultats attendus pour chaque rapport, avec captures d'écran annotées si nécessaire.
- **Code SQL** : Les requêtes SQL complètes, commentées pour expliquer leur logique et leur rôle.
- **Exemples de résultats** : Un exemple de résultat pour chaque requête, illustrant les rapports générés.

15 - Évolution 4 – Python / CSV - Analyse consolidée des logs

- Créez un script Python, intitulé `analyseur_logs.py`, capable de parcourir automatiquement tous les fichiers journaux disponibles, d'en extraire les données pertinentes, et de produire un fichier CSV récapitulatif nommé `statistiques_globales.csv`. Ce fichier devra inclure :
  - Les URL les plus consultées, triées par popularité décroissante.
  - La liste des employés ou utilisateurs les plus actifs, basée sur le nombre de requêtes effectuées.
  - Une analyse des tendances d'activité, regroupées par période (par semaine ou par mois).

  Le programme doit être robuste, bien structuré, et facilement adaptable pour prendre en compte de nouveaux types d'analyses si nécessaire.

Documentation à produire :

- **Compte-rendu :** Résumé des analyses avec captures d'écran annotées et conclusions.
- **Code source Python :** Script complet `analyseur_logs.py`, commenté.
- **Exemple de CSV :** Fichier `statistiques_globales.csv` avec données illustratives.
- **Visualisations :** Graphiques clés avec explications des tendances.

16 - Évolution 5 – Python - Archivage automatisé et sécurisé des journaux

- Concevez un script Python, intitulé `archivage_logs.py`, qui automatise la gestion des fichiers journaux anciens pour optimiser l'utilisation de l'espace disque tout en garantissant la sécurité des données.

Le programme devra :

1. **Identifier les fichiers logs anciens :** Repérer automatiquement les fichiers journaux âgés de plus de 30 jours.
2. **Archiver les fichiers :** Les déplacer vers un répertoire d'archivage spécifique ou les compresser pour réduire leur taille.
3. **Chiffrer les archives :** Appliquer un chiffrement sécurisé avec un algorithme robuste (par exemple, AES-256) pour protéger les données sensibles contenues dans les fichiers journaux.
4. **Tester la restauration :** Mettre en place des tests réguliers pour vérifier que les fichiers archivés peuvent être restaurés et déchiffrés correctement, garantissant ainsi leur intégrité et leur accessibilité future.
5. **Assurer une exécution récurrente :** Configurer le programme pour qu'il fonctionne automatiquement à des intervalles réguliers, sans intervention manuelle.
6. **Enregistrer les opérations :** Générer des journaux d'exécution détaillés retraçant toutes les étapes du processus, y compris les fichiers traités, les éventuelles erreurs rencontrées, et le résultat final de chaque opération.

Documentation à produire :

- **Compte-rendu :** Description des étapes d'archivage, des résultats des tests de restauration, et des captures d'écran des journaux d'exécution annotés. Inclure les principales observations.
- **Code source Python :** Script complet `archivage_logs.py`, structuré et commenté pour expliquer chaque étape du processus (identification, archivage, chiffrement, test).
- **Exemple d'archives :** Exemple d'archive chiffrée (avec des données non sensibles) illustrant le format et les sorties attendues.
- **Logs d'exécution :** Exemple d'un fichier journal détaillant les opérations effectuées avec les éventuelles erreurs ou anomalies détectées.

17 - Évolution 6 – Python - Développement d'une interface graphique

- Réalisez une interface graphique avec Python pour permettre l'interaction et l'exécution d'un script de manière intuitive. Cette interface devra inclure des éléments visuels simples et clairs pour guider l'utilisateur et optimiser l'expérience.

Le programme devra :

1. **Créer une interface graphique :** Utiliser la bibliothèque `tkinter` pour concevoir une fenêtre principale nommée `interface_script.py`, incluant :
   - Des libellés pour guider l'utilisateur.
   - Des champs de saisie pour renseigner les informations nécessaires au fonctionnement du script.
   - Des boutons pour exécuter le script ou accéder à d'autres fonctions (aide, confirmation, etc.).
   - Un menu pour des options complémentaires si nécessaire.
2. **Gérer les interactions :** Intégrer les messages générés par le script directement dans l'interface ou via des pop-ups informatifs.
3. **Prévoir des boîtes de dialogue :** Ajouter des fenêtres contextuelles pour :
   - Demander des informations manquantes.
   - Proposer des corrections en cas d'erreurs.
   - Confirmer des actions importantes.
4. **Inclure une aide sommaire :** Offrir à l'utilisateur une aide succincte sous forme de bouton ou d'option dans un menu, expliquant le fonctionnement de l'interface et du script associé.
5. **Assurer une logique claire :** Organiser les éléments de l'interface pour qu'ils suivent une logique intuitive et facile à comprendre, même pour un non-informaticien.

Documentation à produire :

- **Schéma de l'interface :** Présentation visuelle de la fenêtre principale et des différents éléments de l'interface (boîtes de dialogue, pop-ups, etc.), disposés sur un schéma global avec des flèches pour illustrer la logique de navigation. Ajouter des commentaires pour expliquer chaque élément de manière accessible à un utilisateur novice.
- **Code source Python :** Script complet nommé `interface_script.py`, structuré et commenté pour détailler chaque étape de la création et des fonctionnalités.
- **Compte rendu de test :** Description des tests effectués, incluant les captures d'écran des différentes étapes de l'utilisation de l'interface, les erreurs corrigées, et les résultats obtenus.

18 - ANNEXE 1 - Structure des tables de la base de données logs_web

Table `employes` (SALARIES)

Nom du champ	Type	Description	Contraintes
`id_employe`	INT	Identifiant unique de l'employé	Clé primaire, Auto-incrémentée
`nom`	VARCHAR(50)	Nom de famille	Non nul
`prenom`	VARCHAR(50)	Prénom	Non nul
`email`	VARCHAR(100)	Adresse e-mail	Unique
`adresse_ip`	VARCHAR(15)	Adresse IP de l'employé	Unique
`date_creation`	DATETIME	Date et heure de création de l'enregistrement	Par défaut : NOW()
`date_modif`	DATETIME	Date et heure de la dernière mise à jour	Mise à jour automatique

Table `journaux_acces` (PROXY)

Nom du champ	Type	Description	Contraintes
`id_acces`	INT	Identifiant unique de l'accès	Clé primaire, Auto-incrémentée
`adresse_ip_employe`	VARCHAR(15)	Adresse IP de l'employé (référence)	Clé étrangère vers `employes.adresse_ip`
`horodatage`	DATETIME	Date et heure de l'accès	Non nul
`url_consultee`	TEXT	URL consultée	Non nul
`methode_http`	VARCHAR(10)	Méthode HTTP utilisée (GET, POST, etc.)	Non nul
`code_reponse`	INT	Code de réponse HTTP (ex : 200, 404)	Optionnel
`date_creation`	DATETIME	Date et heure de création de l'enregistrement	Par défaut : NOW()
`date_modification`	DATETIME	Date et heure de la dernière mise à jour	Mise à jour automatique

Justifications des champs `date_creation` et `date_modification` :

- Le champ `date_creation` permet de connaître la date et l'heure exactes de création d'un enregistrement, ce qui est crucial pour suivre l'évolution des données et auditer les activités.
- Le champ `date_modification` garantit une transparence dans les mises à jour des enregistrements en fournissant une information fiable sur la dernière modification.
- Ces champs automatisés facilitent la maintenance et permettent d'identifier les données obsolètes ou erronées.

Exemple de données pour la table `employes`

id_employe	nom	prenom	adresse_email	adresse_ip	date_creation	date_modif
1	DUFONT	Marie	marie.dufont@example.com	192.168.2.2	2024-01-01 10:00:00	2024-01-01 10:00:00
2	DESBOIS	Paul	paul.desbois@example.com	192.168.2.1	2024-01-01 10:05:00	2024-01-01 10:05:00
3	DURANTON	Quentin	quentin.duranton@example.com	192.168.2.3	2024-01-01 10:10:00	2024-01-01 10:10:00
4	LEJAUNE	Laurence	laurence.lejaune@example.com	192.168.2.100	2024-01-01 10:15:00	2024-01-01 10:15:00

Exemple de données pour la table `journaux_acces`

id_acces	adresse_ip_employe	horodatage	url_consultee	methode_http	code_reponse	date_creation	date_modif
1	192.168.2.2	2024-01-02 10:00:00	http://example.com/page1	GET	200	2024-01-02 10:00:00	2024-01-02 10:00:00
2	192.168.2.1	2024-01-02 10:05:00	http://example.com/page2	POST	201	2024-01-02 10:05:00	2024-01-02 10:05:00